Summer of Science (SoS'26)

Image Generation — Mid-term Progress Report

From First Principles to Generative Models

Mathematical Foundations, Variational Autoencoders, and Generative Adversarial Networks

Mentor: Yash Shende
Mentee: Chirag Khandelwal

20th June 2026

Contents

Chapter 1

Mathematical Preliminaries for Generative Modelling

Deep generative models are, at their core, statistical machines: they place a probability distribution over the data and learn its parameters from examples. This section assembles the linear-algebraic and probabilistic vocabulary needed to make that statement precise, and ends with maximum likelihood estimation, the optimisation principle that the rest of the report repeatedly specialises.

1.1 Linear Algebra Foundations

Definition 1.1.1 (Vector space)**.** *A vector space V over a field $\mathbb{F}$ (typically $\mathbb{R}$ in this report) is a set of objects, called vectors, that is closed under addition and scalar multiplication: for all $u, v \in V$ and $\alpha \in \mathbb{F}$,*

$$u + v \in V, \qquad \alpha \cdot v \in V.$$

Intuitively, V is a "playground" in which linear combinations of vectors never leave the set.

Definition 1.1.2 (Basis, span, dimension, subspace)**.** *A* basis *of V is a minimal set of vectors $\{e_1, \ldots, e_n\}$ whose linear combinations reach every vector in V; the coefficients of a vector with respect to a basis are its* coordinates*. The* span *of a set of vectors is the collection of all vectors reachable as linear combinations of that set. The* dimension *of V is the number of vectors in a basis, equivalently the number of independent degrees of freedom available to V. A* subspace *of V is itself a vector space contained in V, and in particular must contain the zero vector.*

Remark 1.1.1. *Every neural-network feature map (an encoder's bottleneck activation, a GAN's noise vector, the per-pixel image tensor) is, formally, simply a point in a finite-dimensional vector space. The structure imposed on this space — via a choice of basis, a metric, or a probability distribution — is precisely what later sections refer to as the* latent space *of a generative model.*

1.1.1 Eigenvalues and Eigenvectors

Definition 1.1.3 (Eigenpair)**.** *Let A be a square matrix. A non-zero vector v is an* eigenvector *of A with* eigenvalue *λ if*

$$Av = \lambda v.$$

Geometrically, A does not rotate v; it only stretches or flips it, by the factor λ.

Proposition 1.1.1 (Characteristic equation)**.** *The eigenvalues of A are exactly the roots of the characteristic polynomial*

$$\det(A - \lambda I) = 0.$$

Proof sketch. If $Av = \lambda v$ for some $v \neq 0$, then $(A - \lambda I)v = 0$, so $A - \lambda I$ has a non-trivial null space and is therefore singular, i.e. $\det(A - \lambda I) = 0$. Conversely, any λ satisfying $\det(A - \lambda I) = 0$ makes $A - \lambda I$ singular, guaranteeing the existence of a non-zero v with $(A - \lambda I)v = 0$. □

Remark 1.1.2. *Eigendecomposition is not merely a linear-algebra exercise: the covariance matrix of a set of encoder outputs can be diagonalised by its eigenvectors, giving the orthogonal directions of greatest variance in a learned latent space (the same idea underlying Principal Component Analysis). This connects directly to the discussion of latent-space structure in Section 2.*

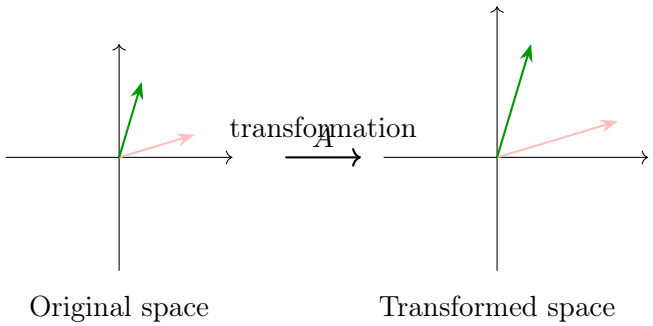

Figure 1.1: Geometric meaning of an eigenpair (v, λ). The pink and green vectors are eigenvectors of A: after the transformation they only scale (by λ), without changing direction.

1.2 Probability Theory and Bayesian Inference

1.2.1 Bayes' Theorem

Definition 1.2.1 (Bayes' theorem)**.** *For a hypothesis H and evidence E,*

$$P(H \mid E) = \frac{P(E \mid H)\, P(H)}{P(E)}.$$

Derivation. By the definition of conditional probability, $P(H, E) = P(H \mid E)P(E) = P(E \mid H)P(H)$. Equating the two right-hand expressions and dividing through by $P(E)$ (assumed non-zero) gives Bayes' theorem directly.

Remark 1.2.1. *The four terms carry standard names that recur throughout this report: $P(H)$ is the* prior *(belief before observing evidence), $P(E \mid H)$ is the* likelihood *(how probable the evidence is under H), $P(H \mid E)$ is the* posterior *(updated belief), and $P(E)$ is the* marginal likelihood*, a normalising constant ensuring the posterior integrates to one: posterior $\propto$ likelihood $\times$ prior. In Section 2, the encoder of a VAE is precisely a learned approximation to an intractable posterior $p(z \mid x)$.*

1.2.2 The Gaussian Distribution

Definition 1.2.2 (Univariate Gaussian)**.** *A random variable x is normally distributed with mean μ and variance σ^2, written $x \sim \mathcal{N}(\mu, \sigma^2)$, if its density is*

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

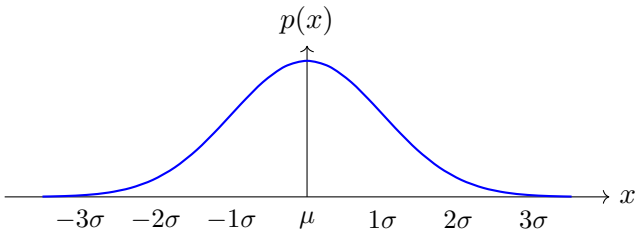

Figure 1.2: The standard Gaussian density. Approximately 68% of the probability mass lies within one standard deviation of the mean, 95% within two, and 99.7% within three.

1.2.3 Kullback–Leibler Divergence

Definition 1.2.3 (KL divergence)**.** *For two distributions P and Q over the same support,*

$$\mathrm{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

(or the corresponding integral for continuous variables). It measures the information lost when Q is used to approximate P.

Proposition 1.2.1 (Non-negativity — Gibbs' inequality)**.** $\mathrm{KL}(P\|Q) \geq 0$ *for all P, Q, with equality if and only if $P = Q$ everywhere.*

Proof sketch. Since $-\log$ is convex, Jensen's inequality gives

$$\mathrm{KL}(P\|Q) = \mathbb{E}_P\left[-\log \frac{Q(x)}{P(x)}\right] \geq -\log \mathbb{E}_P\left[\frac{Q(x)}{P(x)}\right] = -\log \sum_x Q(x) = -\log 1 = 0,$$

with equality exactly when $Q(x)/P(x)$ is almost surely constant, i.e. $P = Q$. □

Remark 1.2.2. *Unlike a true distance, KL divergence is asymmetric:* $\mathrm{KL}(P\|Q) \neq \mathrm{KL}(Q\|P)$ *in general. This asymmetry is not a minor technicality: Section 2 uses the forward KL term* $\mathrm{KL}(q(z \mid x)\|p(z))$ *inside the VAE objective, while Section 3 shows that the GAN's adversarial objective implicitly minimises the symmetrised Jensen–Shannon divergence — the same building block, used in two structurally different ways.*

1.3 Maximum Likelihood Estimation

Definition 1.3.1 (Maximum likelihood estimator)**.** *Given i.i.d. observations $X = \{x_1, \ldots, x_n\}$ drawn from a parametric model $p(x \mid \theta)$, the maximum likelihood estimate is*

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_\theta P(X \mid \theta) = \arg\max_\theta \prod_{i=1}^{n} p(x_i \mid \theta).$$

Because the logarithm is monotonically increasing, this is equivalent to maximising the log-likelihood

$$\ell(\theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta),$$

which trades a numerically unstable product for a numerically convenient sum without changing the location of the maximum.

Proposition 1.3.1 (Five-step MLE procedure)**.** *The maximum likelihood estimate of a parametric model can be found by:*

(1) Choosing a parametric family for the data (e.g. $\mathcal{N}(\mu, \sigma^2)$, $\mathrm{Ber}(p)$, $\mathrm{Pois}(\lambda)$).

(2) Writing the log-likelihood $\ell(\theta) = \sum_i \log p(x_i \mid \theta)$.

(3) Differentiating and setting the gradient to zero: $\partial \ell(\theta)/\partial \theta = 0$.

(4) Solving the resulting equation for θ to obtain a candidate $\hat{\theta}$.

(5) Verifying that $\hat{\theta}$ is a maximum by checking $\partial^2 \ell(\theta)/\partial \theta^2\big|_{\theta=\hat{\theta}} < 0$ (concavity).

Example 1.3.1 (Gaussian MLE)**.** *For data drawn i.i.d. from $\mathcal{N}(\mu, \sigma^2)$, the log-likelihood is*

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2.$$

Figure 1.3: The maximum likelihood estimation pipeline: a likelihood is log-transformed, differentiated, and solved to recover the estimate $\hat{\theta}$.

Differentiating with respect to μ and setting the result to zero gives $\hat{\mu} = \frac{1}{n} \sum_i x_i$, the sample mean. Substituting $\hat{\mu}$ back and repeating the procedure with respect to σ^2 gives $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \hat{\mu})^2$, the (biased) sample variance. A full derivation, including the second-derivative concavity check required by the proposition above, is given in Appendix A.1.

Remark 1.3.1. *Maximum likelihood is not merely a Week-1 exercise: training a VAE by maximising the ELBO (Section 2.2) is best understood as an approximate, tractable form of MLE on $\log p(x)$, and the standard GAN objective (Section 3.1) is shown to correspond, at its optimum, to minimising a likelihood-ratio–based divergence between the real and modelled data distributions. MLE is therefore the thread that ties the remainder of this report together.*

Chapter 2

Variational Autoencoders

2.1 From Deterministic to Probabilistic Autoencoders

Definition 2.1.1 (Autoencoder)**.** *An autoencoder is a pair of neural networks — an encoder $f_\phi : \mathcal{X} \to \mathcal{Z}$ and a decoder $g_\theta : \mathcal{Z} \to \mathcal{X}$ — trained so that $g_\theta(f_\phi(x)) \approx x$, learning a compressed representation $z \in \mathcal{Z}$ of the input x.*

Definition 2.1.2 (Variational Autoencoder, Kingma & Welling)**.** *A Variational Autoencoder (VAE) replaces the deterministic encoder of a standard autoencoder with a stochastic one: instead of mapping x to a single point z, it maps x to the parameters $(\mu, \log \sigma^2)$ of a distribution $q_\phi(z \mid x)$, typically a diagonal Gaussian, from which z is sampled before being passed to the decoder $p_\theta(x \mid z)$.*

Remark 2.1.1. *This probabilistic reformulation yields two properties absent from a standard autoencoder:* continuity *(nearby points in latent space decode to similar images, enabling interpolation) and* completeness *(the latent space is densely populated, so that sampling from the prior anywhere yields a plausible image). Both properties are verified experimentally in Section 2.6.*

The MNIST dataset, used throughout this section, consists of 70,000 greyscale handwritten-digit images of size 28×28, split into 60,000 training and 10,000 test examples; it is the standard benchmark for early-stage generative modelling work.

2.2 The Evidence Lower Bound

Directly maximising $\log p_\theta(x) = \log \int p_\theta(x \mid z)\, p(z)\, dz$ is intractable, since the integral runs over the entire latent space. The variational approach instead introduces a tractable approximate posterior $q_\phi(z \mid x)$ and optimises a lower bound on the log-likelihood.

Proposition 2.2.1 (Evidence Lower Bound)**.** *For any distribution $q_\phi(z \mid x)$,*

$$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\mathcal{L}_{\mathrm{ELBO}}(\theta,\phi;x)},$$

with equality if and only if $q_\phi(z \mid x) = p_\theta(z \mid x)$, the true posterior.

Proof. Starting from the definition of the marginal likelihood and multiplying and dividing by

$q_\phi(z \mid x)$,

$$\begin{aligned}
\log p_\theta(x) &= \log \int p_\theta(x,z)\,dz = \log \mathbb{E}_{q_\phi(z|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \\
&\geq \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \quad \text{(Jensen's inequality, as above)} \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(z)}{q_\phi(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) \\
&= \mathcal{L}_{\mathrm{ELBO}}(\theta, \phi; x).
\end{aligned}$$

Equivalently, one can show $\log p_\theta(x) = \mathcal{L}_{\mathrm{ELBO}}(\theta,\phi;x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$; since the KL term is non-negative and vanishes only when $q_\phi(z \mid x) = p_\theta(z \mid x)$, the bound and the equality condition follow immediately. □

Remark 2.2.1. *This recasts VAE training as a direct descendant of the maximum likelihood principle of Section 1.3: maximising $\mathcal{L}_{\mathrm{ELBO}}$ with respect to θ is equivalent to maximising a lower bound on the log-likelihood $\log p_\theta(x)$ — variational inference is, in this precise sense, approximate MLE. Maximising the same bound with respect to ϕ simultaneously tightens it, pulling $q_\phi(z \mid x)$ towards the true posterior.*

The negative ELBO, used as the training loss, decomposes into a reconstruction term and a regularisation term,

$$\mathcal{L}(\theta,\phi;x) = \underbrace{-\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right]}_{\mathcal{L}_{\mathrm{recon}}} + \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\mathcal{L}_{\mathrm{KL}}}.$$

2.3 The Reparameterisation Trick

Optimising $\mathcal{L}_{\mathrm{ELBO}}$ by gradient descent requires differentiating through the sampling step $z \sim q_\phi(z \mid x)$, which is not directly differentiable.

Definition 2.3.1 (Reparameterisation)**.** *The reparameterisation trick rewrites the sampling operation as a deterministic, differentiable function of ϕ and an independent noise source:*

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Proposition 2.3.1. *Gradients of $\mathbb{E}_{q_\phi(z|x)}[h(z)]$ with respect to ϕ, for any differentiable h, can be estimated without bias by sampling $\varepsilon \sim \mathcal{N}(0,I)$ once and backpropagating through $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$.*

Proof sketch. Since ε does not depend on ϕ, the expectation can be rewritten over the fixed distribution of ε rather than the ϕ-dependent distribution of z: $\mathbb{E}_{q_\phi(z|x)}[h(z)] = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,I)}\left[h(\mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon)\right]$. Differentiation under the expectation (valid for smooth h, μ_ϕ, σ_ϕ) can now pass directly through μ_ϕ and σ_ϕ, exactly as for any other deterministic layer. □

Remark 2.3.1. *The network is parameterised to output $\log \sigma^2$ rather than σ directly: this keeps the raw network output unconstrained over $\mathbb{R}$, while $\sigma = \exp(\frac{1}{2}\log\sigma^2)$ is automatically guaranteed to be positive.*

Listing 2.1: Reparameterisation trick in PyTorch.

```
def reparameterize(self, mu, log_var):
    """
    z = mu + std * epsilon, epsilon ~ N(0, I)
    """
    std = torch.exp(0.5 * log_var)   # sigma = exp(log_sigma^2 / 2)
    eps = torch.randn_like(std)      # epsilon ~ N(0, I)
    return mu + eps * std
```

2.4 Architecture and Implementation

The implemented VAE consists of three components — an encoder, the reparameterisation operation, and a decoder — bundled into a single `nn.Module`. Both encoder and decoder are simple fully-connected networks, summarised in Tables 2.1 and 2.2.

Table 2.1: Encoder architecture.

Layer	Dimensions	Activation
Input	784	—
Fully connected	400	ReLU
μ output	2	—
$\log \sigma^2$ output	2	—

Table 2.2: Decoder architecture.

Layer	Dimensions	Activation
Input (latent)	2	—
Fully connected	400	ReLU
Output	784	Sigmoid

The decoder's sigmoid output layer constrains pixel values to $[0, 1]$, matching the normalised inputs described in Section 2.5 below.

Listing 2.2: VAE model definition.

```
class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=2):
        super(VAE, self).__init__()

        # Encoder layers
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)    # Mean (mu)
        self.fc_var = nn.Linear(hidden_dim, latent_dim)   # Log-variance

        # Decoder layers
        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)

```

```
    def encode(self, x):
        h = F.relu(self.fc1(x))
        return self.fc_mu(h), self.fc_var(h)  # (mu, log_var)

    def decode(self, z):
        h = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h))  # pixels in [0, 1]

    def forward(self, x):
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        return self.decode(z), mu, log_var
```

2.4.1 The Closed-Form KL Term

Both terms of the negative ELBO require concrete formulas to be implemented. The reconstruction term is taken to be the binary cross-entropy between input and reconstruction, treating each pixel as an independent Bernoulli variable:

$$\mathcal{L}_{\text{recon}} = -\sum_{i} \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right].$$

Proposition 2.4.1 (KL term for a diagonal Gaussian posterior)**.** *If* $q_\phi(z \mid x) = \mathcal{N}(\mu, \operatorname{diag}(\sigma^2))$ *and the prior is* $p(z) = \mathcal{N}(0, I)$*, then*

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) = -\frac{1}{2} \sum_{j} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

A full derivation of this expression, generalised first to two arbitrary Gaussians and then specialised to the standard normal prior, is given in Appendix A.2.

Listing 2.3: VAE loss function: reconstruction plus closed-form KL term.

```
def loss_function(recon_x, x, mu, log_var):
    """
    Total VAE loss = Reconstruction (BCE) + KL Divergence.
    """
    # Sum over pixels; mean handled by the optimiser step
    BCE = F.binary_cross_entropy(recon_x, x, reduction='sum')

    # -0.5 * sum(1 + log(sigma^2) - mu^2 - sigma^2)
    KLD = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())

    return BCE + KLD
```

2.5 Training Methodology

The MNIST dataset was loaded via `torchvision`, normalised to $[0, 1]$, and flattened to length-784 vectors, since the encoder operates on flat inputs rather than 28×28 image tensors.

Listing 2.4: Dataset loading and preprocessing.

```
transform = transforms.Compose([
    transforms.ToTensor()  # Converts pixel values to [0, 1]
])

train_dataset = datasets.MNIST(root='./data', train=True,
    transform=transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False,
    transform=transform, download=True)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)
```

The training loop optimises the combined loss with the Adam optimiser, accumulating the per-batch loss into an average reported once per epoch:

Table 2.3: Hyperparameters used for VAE training.

Hyperparameter	Value
Framework	PyTorch
Dataset	MNIST (60,000 train / 10,000 test)
Optimiser	Adam
Learning rate	0.001
Batch size	128
Epochs	20
Latent dimension	2
Input dimension	784 (28×28)
Hidden dimension	400

Listing 2.5: VAE training loop.

```
model = VAE(input_dim=784, hidden_dim=400, latent_dim=2).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def train(epoch):
    model.train()
    train_loss = 0
    for batch_idx, (data, _) in enumerate(train_loader):
        data = data.view(-1, 784).to(device)
        optimizer.zero_grad()

        recon_batch, mu, log_var = model(data)
        loss = loss_function(recon_batch, data, mu, log_var)

        loss.backward()
        train_loss += loss.item()
        optimizer.step()

    avg_loss = train_loss / len(train_loader.dataset)
    return avg_loss
```

The training loss decreased steadily across all 20 epochs, confirming stable convergence of the joint ELBO objective, with no instabilities of the kind later observed for the adversarial training of Section 3.

2.6 Experimental Results

Four experiments were used to evaluate the trained model: the training loss curve, a 2D projection of the latent space, unconditional random sampling, and latent-space interpolation. Plots from the corresponding experiment notebook are referenced below as placeholders for direct insertion of the generated figures.

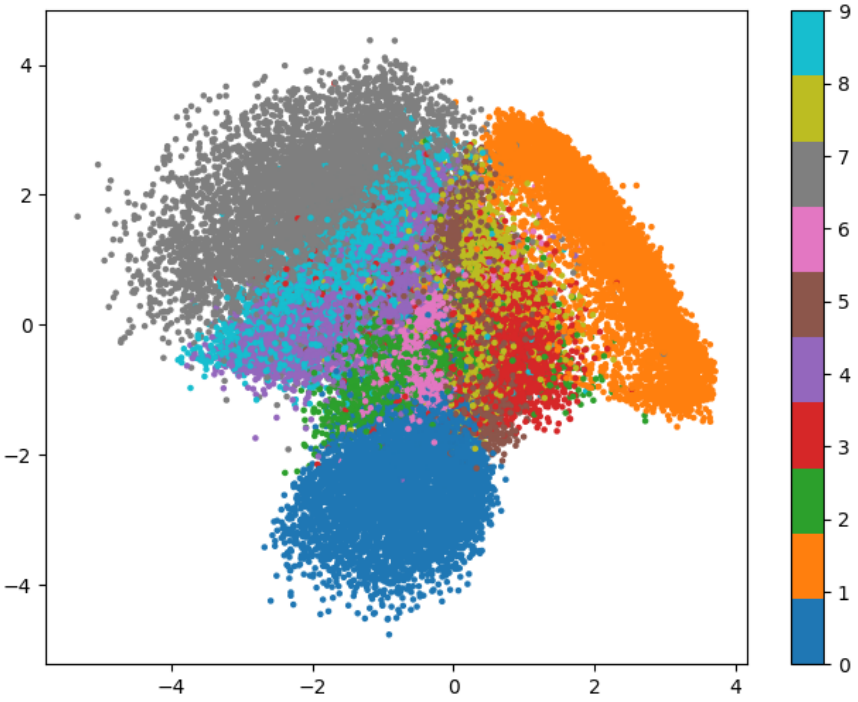

Figure 2.1: 2D latent space coloured by digit class. Digits of the same class form distinct, spatially coherent clusters.

Listing 2.6: Generating unconditional samples from the prior.

```
def generate_samples(model, num_samples=20, latent_dim=2, device='cpu'):
    model.eval()
    with torch.no_grad():
        z = torch.randn(num_samples, latent_dim).to(device)
        samples = model.decode(z).cpu().view(num_samples, 1, 28, 28)
    return samples
```

Listing 2.7: Latent space interpolation between two encoded digits.

```
def interpolate(model, img1, img2, steps=10, device='cpu'):
    model.eval()
    with torch.no_grad():
        mu1, _ = model.encode(img1.view(1, -1).to(device))
```

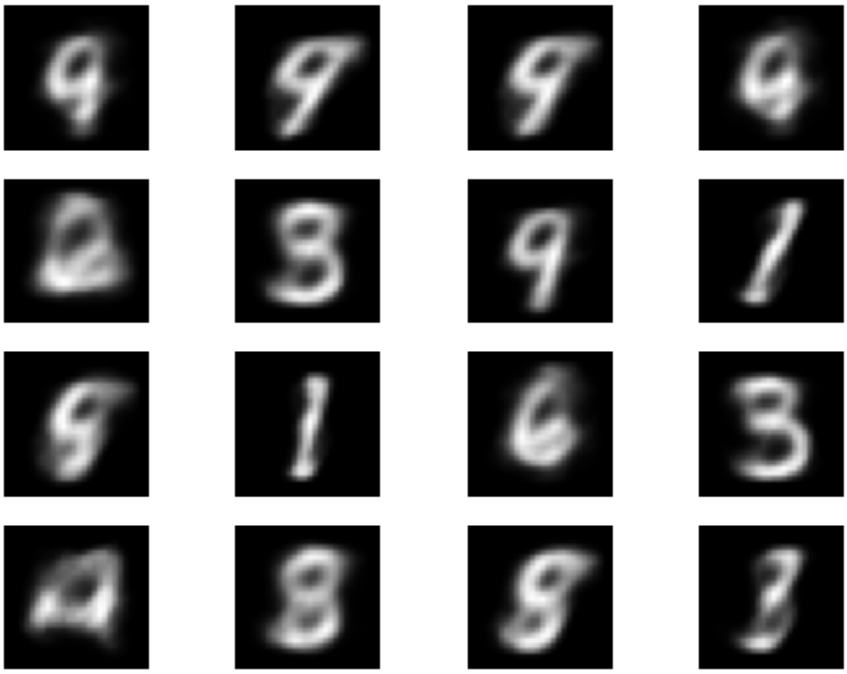

Figure 2.2: Digits generated by decoding random samples from the prior, demonstrating dense and meaningful coverage of the latent space.

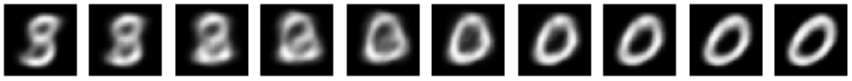

Figure 2.3: Smooth morphological transition between two encoded digit representations, obtained by linearly interpolating $z_t = (1-t)\mu_1 + t\mu_2$ for $t \in \{0, 0.1, \ldots, 1.0\}$.

```
        mu2, _ = model.encode(img2.view(1, -1).to(device))

        interpolated = []
        for t in torch.linspace(0, 1, steps):
            z = (1 - t) * mu1 + t * mu2  # linear interpolation
            decoded = model.decode(z).cpu()
            interpolated.append(decoded.view(1, 28, 28))
    return interpolated
```

Chapter 3

Generative Adversarial Networks

3.1 The Adversarial Framework

Where the VAE of Chapter 2 defines an explicit probabilistic model and optimises a tractable lower bound on its likelihood, a Generative Adversarial Network (GAN) defines a generative process implicitly, through a two-player game between a generator and a discriminator, and never writes down a likelihood at all.

Definition 3.1.1 (GAN minimax objective)**.** *Let G (the generator) map noise $z \sim p_z$ to synthetic data $G(z)$, and let D (the discriminator) map data x to an estimated probability $D(x) \in (0,1)$ that x is real. G and D are trained jointly on the value function*

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}\left[\log D(x)\right] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D(G(z))\right)\right].$$

Proposition 3.1.1 (Optimal discriminator)**.** *For a fixed generator G, with induced data distribution p_g, the discriminator that maximises $V(D,G)$ is*

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Proof sketch. For fixed G, $V(D,G) = \int p_{\text{data}}(x)\log D(x) + p_g(x)\log(1-D(x))\,dx$. For each x, the integrand has the form $a\log y + b\log(1-y)$ with $a = p_{\text{data}}(x)$, $b = p_g(x)$, which is maximised over $y \in (0,1)$ at $y = a/(a+b)$, giving the stated expression for $D^*(x)$. □

Proposition 3.1.2 (Global optimum)**.** *Substituting D^* into $V(D,G)$, the generator's objective becomes, up to an additive constant, the Jensen–Shannon divergence between p_{data} and p_g:*

$$V(D^*,G) = 2\,\text{JSD}(p_{\text{data}}\|p_g) - \log 4, \qquad \text{JSD}(P\|Q) = \tfrac{1}{2}\text{KL}(P\|M) + \tfrac{1}{2}\text{KL}(Q\|M), \quad M = \tfrac{1}{2}(P+Q),$$

which is minimised, with value $-\log 4$, if and only if $p_g = p_{\text{data}}$.

Remark 3.1.1. *This closes the loop opened in Section 2.2: both the VAE and the GAN are, at their theoretical optimum, minimising a divergence built from the same KL functional — the VAE minimises a forward-KL-flavoured bound with respect to an explicit posterior, while the (vanilla) GAN minimises a symmetrised, Jensen–Shannon version with respect to an implicit one. In practice, this theoretical equilibrium is rarely reached exactly: G and D are optimised by alternating, non-convex gradient steps rather than by jointly solving for the saddle point, which is the root cause of the training instabilities observed in Section 3.5.*

3.2 Deep Convolutional GAN (DCGAN) Architecture

The implemented model follows the DCGAN architecture, replacing the fully-connected layers of a vanilla GAN with convolutional ones, which are far better suited to image data.

3.2.1 Generator: The Artist

The generator starts from a random noise vector and progressively upsamples it into a full-resolution image, acting as an artist sculpting a face out of static.

- **Transposed convolution** (`ConvTranspose2d`): performs the upsampling, expanding a small input into a larger feature map layer by layer — conceptually the inverse of an ordinary convolution.
- **Batch normalisation**: normalises each layer's outputs, stabilising and accelerating training.
- **ReLU**: the activation used in hidden layers, allowing the network to learn complex non-linear structure.
- **Tanh**: used at the output layer to squash pixel values into $[-1, 1]$, matching the normalised range of the real training images.

3.2.2 Discriminator: The Detective

The discriminator inspects an image and estimates the probability that it is real, acting as a detective who must learn to spot ever more convincing forgeries.

- **Convolution** (`Conv2d`): performs downsampling, compressing the image into a smaller set of discriminative features.
- **Batch normalisation**: as in the generator, stabilises intermediate activations.
- **LeakyReLU**: similar to ReLU but allows a small non-zero gradient for negative inputs, preventing "dead" neurons and keeping gradients flowing.
- **Sigmoid**: produces the final scalar probability $D(x) \in (0, 1)$.

Listing 3.1: Representative DCGAN generator block (PyTorch).

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100, feature_maps=64, channels=3):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(latent_dim, feature_maps * 8, 4, 1, 0, bias
                =False),
            nn.BatchNorm2d(feature_maps * 8),
            nn.ReLU(True),
            nn.ConvTranspose2d(feature_maps * 8, feature_maps * 4, 4, 2,
                1, bias=False),
            nn.BatchNorm2d(feature_maps * 4),
            nn.ReLU(True),
            nn.ConvTranspose2d(feature_maps * 4, feature_maps * 2, 4, 2,
                1, bias=False),
```

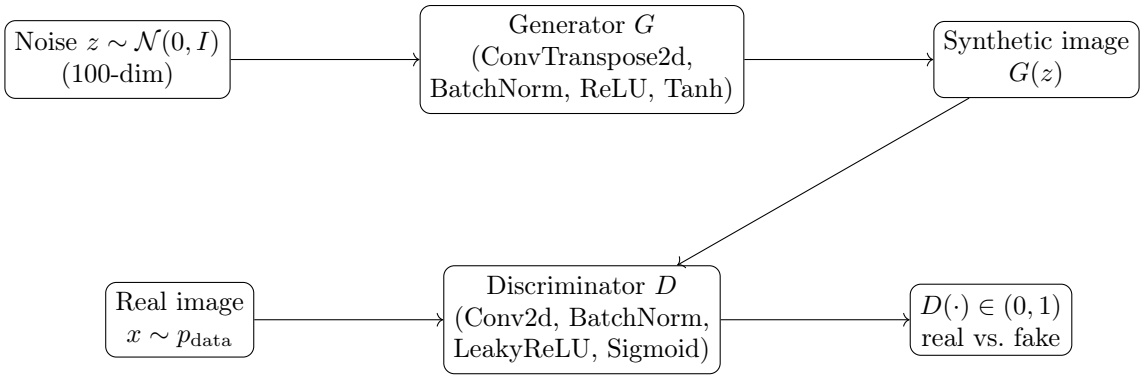

Figure 3.1: The adversarial pipeline: the generator maps noise to a synthetic image, and the discriminator is trained to distinguish such synthetic images from real ones, with the resulting gradient signal used to improve the generator in turn.

```
            nn.BatchNorm2d(feature_maps * 2),
            nn.ReLU(True),
            nn.ConvTranspose2d(feature_maps * 2, feature_maps, 4, 2, 1,
                bias=False),
            nn.BatchNorm2d(feature_maps),
            nn.ReLU(True),
            nn.ConvTranspose2d(feature_maps, channels, 4, 2, 1, bias=False
                ),
            nn.Tanh()
        )

    def forward(self, z):
        return self.main(z)
```

Listing 3.2: Representative DCGAN discriminator block (PyTorch).

```
class Discriminator(nn.Module):
    def __init__(self, feature_maps=64, channels=3):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(channels, feature_maps, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(feature_maps, feature_maps * 2, 4, 2, 1, bias=False)
                ,
            nn.BatchNorm2d(feature_maps * 2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(feature_maps * 2, feature_maps * 4, 4, 2, 1, bias=
                False),
            nn.BatchNorm2d(feature_maps * 4),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(feature_maps * 4, feature_maps * 8, 4, 2, 1, bias=
                False),
            nn.BatchNorm2d(feature_maps * 8),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(feature_maps * 8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )
```

```
19
20      def forward(self, x):
21          return self.main(x).view(-1, 1).squeeze(1)
```

3.3 Dataset: CelebA

The CelebA (CelebFaces Attributes) dataset comprises more than 200,000 celebrity face images, each annotated with facial-attribute labels. For this project, all images were resized to 64×64 pixels to keep training computationally manageable, and pixel values were normalised to $[-1, 1]$ to match the range of the generator's Tanh output.

3.4 Training Configuration

Table 3.1: DCGAN training hyperparameters.

Hyperparameter	Value
Batch size	128
Learning rate	0.0002
Latent dimension	100
Optimiser	Adam
Epochs	25–50

3.5 Experimental Results

During training, the generator progressively learned coherent facial structure, evolving from undifferentiated noise toward images with recognisable eyes, noses, and facial outlines, while the discriminator simultaneously became harder to fool, learning increasingly subtle distinguishing cues. Generated faces grew visibly sharper in later epochs, with finer detail and more realistic texture than the blurry outputs typical of early training. The loss curves below show occasional sharp spikes and sustained fluctuation rather than smooth convergence; this is a common and largely expected characteristic of adversarial training, consistent with the non-convex, alternating-optimisation picture above, and does not by itself indicate failure to learn.

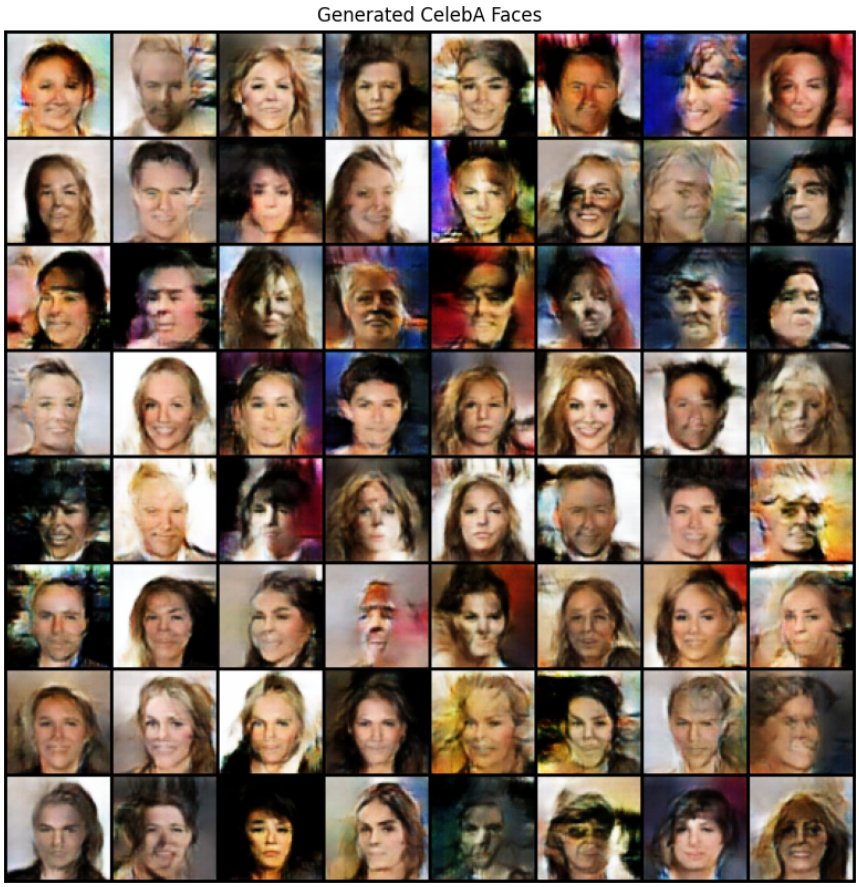

Figure 3.2: Sample face images generated by the trained DCGAN. Faces show recognisable, if imperfect, structure: consistent eye, nose, and hairline placement, with some residual texture and colour artefacts characteristic of a moderately-trained DCGAN at 64×64 resolution.

Chapter 4

Comparative Analysis: Explicit versus Implicit Generative Modelling

Having implemented both paradigms, it is useful to make explicit the trade-offs that the preceding theory predicts.

These differences are not incidental but follow directly from the two divergence measures identified in Sections 2.2 and 3.1. Minimising $\mathrm{KL}(q\|p)$, as the VAE does, penalises q heavily for placing mass where p has none, which tends to produce *mode-covering* but blurrier samples, since the model must hedge over multiple plausible reconstructions. Minimising a Jensen–Shannon-flavoured objective, as the GAN does at its theoretical optimum, more readily tolerates ignoring some modes of p_{data} in exchange for confidently sharp samples on the modes it does cover — a tendency consistent with the sharper, but not perfectly diverse, faces generated by the DCGAN.

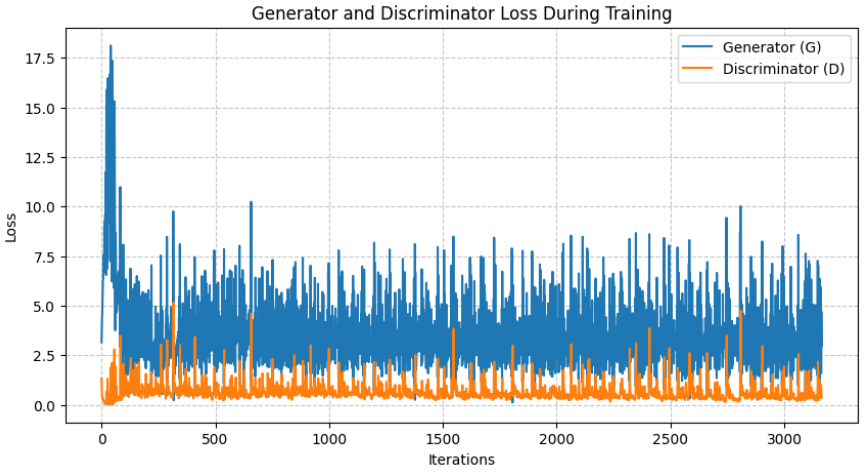

Figure 4.1: Generator and discriminator loss over training iterations. The discriminator loss remains low and comparatively stable, while the generator loss is consistently higher and considerably noisier, oscillating rather than monotonically decreasing—the expected signature of adversarial, minimax optimisation rather than a single descending objective.

Conclusion

This report has traced a single arc from mathematical first principles to two working deep generative models. Linear algebra supplied the language of eigendecomposition used to describe latent-space structure; Bayesian inference and the Kullback–Leibler divergence supplied the vocabulary in which both the VAE's ELBO and the GAN's adversarial objective are most naturally expressed; and maximum likelihood estimation supplied the optimisation principle that both models, in their respective explicit and implicit ways, approximate.

On top of this foundation, a Variational Autoencoder was implemented from scratch in PyTorch and trained on MNIST, with its loss function derived rather than assumed: the reconstruction term from a Bernoulli observation model, and the KL regularisation term in closed form for a diagonal Gaussian posterior against a standard normal prior. The resulting model learned a smooth, well-clustered two-dimensional latent space supporting both unconditional generation and meaningful interpolation. A Deep Convolutional GAN was then implemented and trained on the CelebA face dataset, its minimax objective shown to correspond, at its theoretical optimum, to minimisation of the Jensen–Shannon divergence between real and generated data distributions — and its empirically noisier, less stable training dynamics shown to be a direct, expected consequence of that theory rather than an implementation flaw.

The mathematical intuition built across this work — that generative models are, at heart, exercises in approximating an intractable likelihood or a related divergence by a tractable surrogate — provides a natural foundation for further study. Promising directions include conditional and β-disentangled VAEs, which extend the ELBO framework developed in Section 2.2 to controllable and interpretable generation; Wasserstein GANs, which replace the Jensen–Shannon objective of Proposition 3.2 with the Earth Mover's distance to improve training stability; and score-based / diffusion models, which sidestep both the variational bound and the adversarial game in favour of learning the gradient of the log-density directly. Each of these directions can be understood, in light of this report, as a different choice of which divergence to approximate, and how.

Table 4.1: Qualitative comparison of the VAE (Chapter 2) and DCGAN (Chapter 3) implementations studied in this report.

	Variational Autoencoder	**DCGAN**
Likelihood	Explicit, via a tractable lower bound (ELBO)	Implicit; no likelihood is ever computed
Training objective	Single objective, jointly minimised	Two competing objectives, alternately optimised
Training stability	Stable, monotonically decreasing loss	Frequently unstable, oscillating losses
Sample sharpness	Comparatively blurry, a known consequence of pixel-wise reconstruction loss	Comparatively sharp, especially in later training epochs
Latent space	Smooth and structured by construction, via the KL regulariser	Not explicitly regularised; structure is an emergent property
Divergence minimised at optimum	Forward-KL-flavoured bound	Jensen–Shannon divergence
Mode coverage	Encouraged by the explicit posterior over every input	No explicit guarantee; susceptible to mode collapse

Bibliography

[1] Kingma, D. P. and Welling, M. (2013). *Auto-Encoding Variational Bayes*. arXiv:1312.6114.

[2] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). *Generative Adversarial Networks*. Advances in Neural Information Processing Systems (NeurIPS).

[3] Radford, A., Metz, L., and Chintala, S. (2015). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. arXiv:1511.06434.

[4] LeCun, Y., Cortes, C., and Burges, C. J. C. *The MNIST Database of Handwritten Digits*.

[5] Liu, Z., Luo, P., Wang, X., and Tang, X. (2015). *Deep Learning Face Attributes in the Wild* (CelebA dataset). International Conference on Computer Vision (ICCV).

[6] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

[7] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.